Open Access | https://doi.org/10.25080/ycyj7495



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

alphagenome-pt: Flexible Training of AlphaGenome Models in PyTorch

Rylie Weaver[1,2], **Gomathi Lakshmanan**[1,2], and **John Lagergren**[2]

[1]University of Tennessee, Knoxville, [2]Oak Ridge National Laboratory

Abstract

We present `alphagenome-pt`, an open-source PyTorch implementation of Google DeepMind's AlphaGenome model for training and fine-tuning. AlphaGenome predicts functional genomic outputs directly from DNA sequence, but the released models are trained on human and mouse genomes, motivating dataset-specific training for other species and biological contexts. `alphagenome-pt` lowers the software barrier to adapting AlphaGenome by porting the JAX model to PyTorch and exposing relevant model hyperparameters to custom configuration.

Keywords genomics, DNA language modeling, deep learning

1. Introduction

We present `alphagenome-pt` (PyPI, GitHub), an open-source PyTorch implementation of Google DeepMind's AlphaGenome model architecture for training and fine-tuning. This paper describes `alphagenome-pt` version 0.4.0, which is used for all examples and validation results. DNA language models such as AlphaGenome learn representations of DNA sequence and can be broadly grouped into two categories based on their prediction objectives: (1) masked language modeling (MLM), which tasks the model with predicting the original DNA sequence given a corrupted input [1], [2], [3], [4], [5], [6], [7], [8], and (2) sequence-to-function modeling, which tasks the model with predicting experimental measurements of biological quantities [9], [10], [11], [12], [13], [14], [15].[1] Example functional prediction tasks include gene expression, chromatin accessibility, splicing, and chromatin contact maps, each of which captures a different aspect of genome function. By serving as computational surrogates for physical experiments, these models provide fast estimates for quantities that would otherwise be relatively expensive and time-consuming to measure. Moreover, DNA sequence inputs can be precisely specified and systematically modified in silico, enabling controlled perturbations that are difficult or impossible to perform experimentally.

Functional measurements are useful in downstream biological workflows. Consequently, sufficiently inexpensive and accurate predictions of those measurements are similarly useful, but sequence-to-function prediction remains a central challenge in genomics because of the complexity of the relationships that must be modeled. AlphaGenome is a DNA language model that advances sequence-to-function prediction by modeling a wide range of functional genomic outputs directly from DNA sequence for human and mouse genomes [9]. However, Google DeepMind's released AlphaGenome models are trained on human and mouse genomes, limiting their direct application to other species. Practitioners who want to apply AlphaGenome to other species may therefore need to train or fine-tune models with the same core architecture on species-specific data. At the same time, the released

[1]The two DNA language model categories are not mutually exclusive since prediction tasks can be swapped while keeping the majority of the model the same. In particular, it's common and often useful to pretrain DNA language models with the MLM task before fine-tuning on sequence-to-function tasks. However, the separation is a useful framework for understanding the differences between DNA language models at a high level.

Published Jul 13, 2026

Correspondence to
Rylie Weaver
rweave15@vols.utk.edu

Open Access

Copyright © 2026 Weaver *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

implementation is written in JAX [16], while many deep learning workflows in the scientific Python community are built in PyTorch [17]. `alphagenome-pt` addresses this gap by allowing researchers to instantiate, modify, train, and checkpoint AlphaGenome models in PyTorch without reimplementing the model themselves. By lowering the software barrier to training and fine-tuning AlphaGenome, `alphagenome-pt` aims to make the model easier to adapt to new biological settings.

2. Goals and Motivations

The goal of `alphagenome-pt` is to make the AlphaGenome model easily trainable in PyTorch, with users only needing to know minimal details about the architecture.

Fine-tuning is a well-established and common method to adapt and/or specialize deep learning models to specific domains or tasks. Fine-tuning is useful for language models generally [3], [4], [5], [18], [19], [20], [21], but is especially relevant for the AlphaGenome architecture because the model allocates a significant number of parameters that are specific to organisms and output tracks. These include organism embedding parameters of shape `[num_organisms, num_channels]` and multiorganism linear layers of shape `[num_organisms, num_in_channels, num_out_channels]` used in the output heads[2]. In the case of the multiorganism linear layers for some output heads (e.g. RNA-Seq gene expression), different output channels may correspond to different biological contexts. This architectural choice is grounded in the biological reality, where the same DNA sequence can produce different measured values (e.g. expression, chromatin accessibility, splicing, contact maps) depending on biological context (e.g. tissue, cell type, developmental stage, environment, and organism).

However, this may present a problem when applying the model to other species or biological contexts. In such cases, a user has three options: (1) Initialize the new parameters randomly, which gives no useful prior and, in the case of the multiorganism linear layers, random output predictions. (2) Reuse the closest existing organism or track, which may be reasonable for closely related settings but becomes weaker as the biological context diverges. (3) Finetune the model on the new species/contexts.

Although future AlphaGenome releases may incorporate more species and biological contexts, it is difficult for any released model to cover every species and context of interest. In many cases, the best solution may be to train or fine-tune AlphaGenome on user-specific data. `alphagenome-pt` is designed to make that option practical.

3. AlphaGenome

AlphaGenome is a hybrid convolutional-transformer architecture that balances long-context with computational feasibility while supporting single-base-pair resolution in its predictions. At a high level, AlphaGenome can be viewed as a U-Net-like encoder-decoder model [22] with transformer layers [23] in the middle.

[2] Parameter shapes are expressed up to permutation.

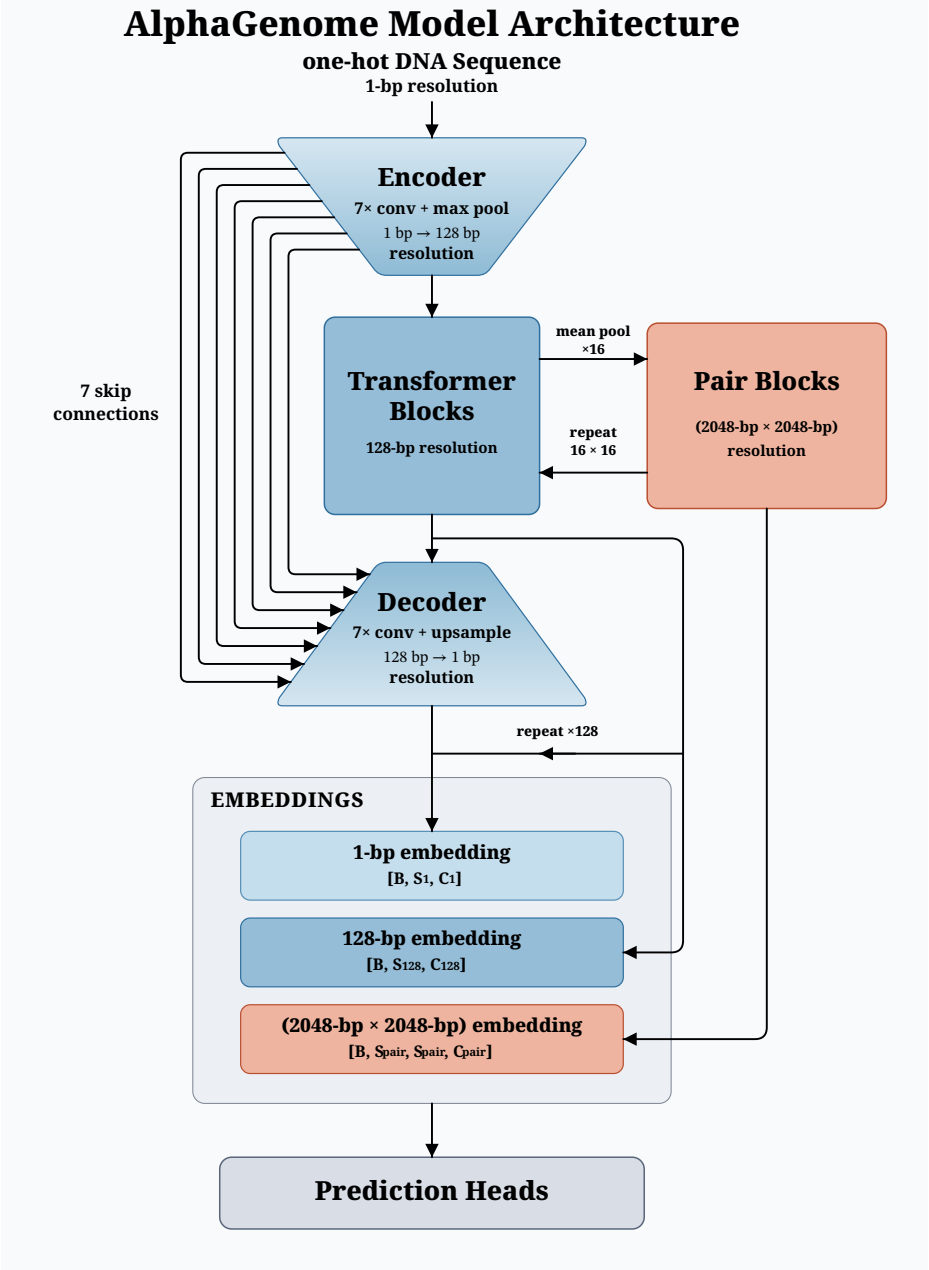


Figure 1. *Overall encoder-transformer-decoder architecture of the AlphaGenome model.*

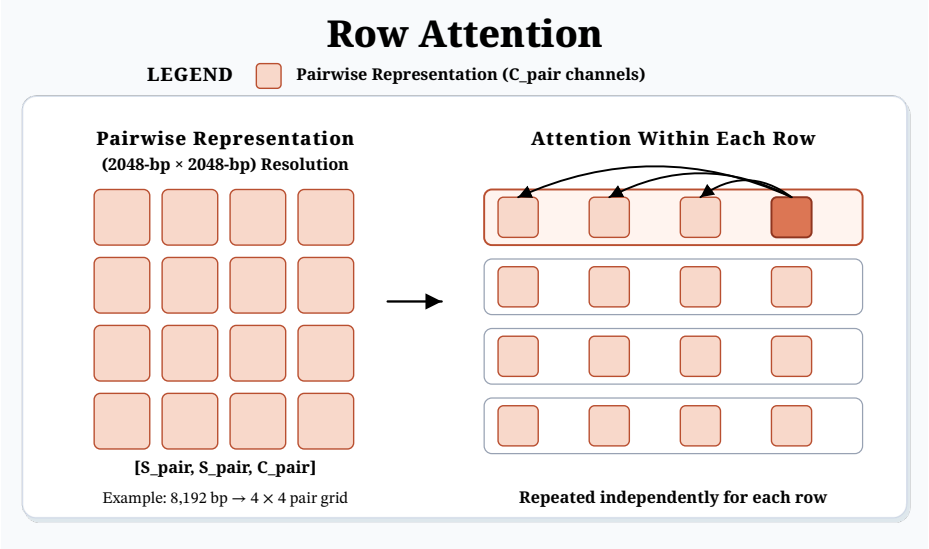


Figure 2. *Row attention mechanism used to update AlphaGenome's pairwise sequence representations.*

Symbol	Meaning	Derivation	Published Value
B	Batch size	User selected	—
S_1	Number of 1-bp positions	$S_1 = S$	1,048,576
S_{128}	Number of 128-bp positions	$S_{128} = S/128$	8,192
S_{pair}	Number of 2,048-bp bins along each pairwise axis	$S_{\text{pair}} = S/2048$	512
C_1	Width of the 1-bp embedding	$M'C$	1,536
C_{128}	Width of the 128-bp embedding	$M'(C + 6I)$	3,072
C_{pair}	Width of the pairwise embedding	`pair_channels`	128

Here, S is `max_seq_len`, C is `num_channels`, I is `channel_increment`, and M' is `embedder_mlp_ratio`.

The model begins with an encoder that consists of seven convolutional downsampling blocks, which capture short-range dependencies and coarsen the sequence from 1-bp to 128-bp resolution. The resulting 128-bp embeddings are then input to transformer layers [23], which capture long-range dependencies over the full context at reduced sequence length. The transformer attention logits are adjusted by an attention bias computed from pairwise sequence representations at a coarser (2048×2048) bp resolution. These pairwise representations are updated with a pair-to-pair attention mechanism (called row attention in AlphaGenome), which improves the model's capability to capture higher-order relationships. Finally, AlphaGenome applies a decoder that consists of seven convolutional upsampling blocks, which capture short-range dependencies and restore a 1-bp resolution representation. U-Net-style skip connections [22] with the fine-grained encoder representations are used to prevent information loss.

The resulting 1-bp, 128-bp, and (2048×2048) bp representations are then passed to task-specific output heads. In total, the encoder-transformer-decoder structure allows AlphaGenome to utilize long-range context while retaining single-base-pair resolution for applicable tasks.

4. Use Cases and Design Philosophy

The core use case of `alphagenome-pt` is to allow users to train the AlphaGenome model on their own datasets to predict functional genomic tracks of interest, including RNA-seq,

CAGE, PRO-cap, ATAC-seq, DNase-seq, transcription factor ChIP-seq, histone-mark ChIP-seq, splice-site classification, splice-site usage, splice-junction prediction, and chromatin contact maps.

4.1. Inference

Load the published checkpoint and return both predictions and embeddings:

```
import torch
from alphagenome_pt import deepmind_model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = deepmind_model(load_state=True, device=device)
sequence = "ACGT" * 512  # 2,048 bp

# Heads can be enabled or disabled before inference, for example:
# model.metadata.metadata["heads"]["splice_sites_junction"]["enabled"] = False

model.eval()
# NOTE: model.predict() disables grad-tracking
predictions, embeddings = model.predict(
    sequence,
    organism_index=0,    # human in the published metadata
    return_embeddings=True,
)
print("embeddings")
print(f"\tembeddings_1bp: {tuple(embeddings.embeddings_1bp.shape)}")
print(f"\tembeddings_128bp: {tuple(embeddings.embeddings_128bp.shape)}")
print(f"\tembeddings_pair: {tuple(embeddings.embeddings_pair.shape)}\n")

for head, outputs in predictions.items():
    print(head)
    for name, value in outputs.items():
        print(f"\t{name}: {tuple(value.shape)}")
```

Full Printed Output

```
embeddings
  embeddings_1bp: (1, 2048, 1536)
  embeddings_128bp: (1, 16, 3072)
  embeddings_pair: (1, 1, 1, 128)

atac
  scaled_predictions_1bp: (1, 2048, 256)
  predictions_1bp: (1, 2048, 256)
  scaled_predictions_128bp: (1, 16, 256)
  predictions_128bp: (1, 16, 256)
dnase
  scaled_predictions_1bp: (1, 2048, 384)
  predictions_1bp: (1, 2048, 384)
  scaled_predictions_128bp: (1, 16, 384)
  predictions_128bp: (1, 16, 384)
procap
  scaled_predictions_1bp: (1, 2048, 128)
  predictions_1bp: (1, 2048, 128)
  scaled_predictions_128bp: (1, 16, 128)
  predictions_128bp: (1, 16, 128)
cage
  scaled_predictions_1bp: (1, 2048, 640)
  predictions_1bp: (1, 2048, 640)
  scaled_predictions_128bp: (1, 16, 640)
  predictions_128bp: (1, 16, 640)
rna_seq
  scaled_predictions_1bp: (1, 2048, 768)
  predictions_1bp: (1, 2048, 768)
  scaled_predictions_128bp: (1, 16, 768)
  predictions_128bp: (1, 16, 768)
chip_tf
  scaled_predictions_128bp: (1, 16, 1664)
  predictions_128bp: (1, 16, 1664)
chip_histone
  scaled_predictions_128bp: (1, 16, 1152)
  predictions_128bp: (1, 16, 1152)
contact_maps
  predictions: (1, 1, 1, 28)
splice_sites_classification
  logits: (1, 2048, 5)
  predictions: (1, 2048, 5)
splice_sites_usage
  logits: (1, 2048, 734)
  predictions: (1, 2048, 734)
splice_sites_junction
  predictions: (1, 512, 512, 734)
  splice_site_positions: (1, 4, 512)
  splice_junction_mask: (1, 512, 512, 734)
```

4.2. Model Instantiation

4.2.1. *Published Architecture*

The published metadata, architecture, and converted checkpoint can be loaded directly with:

```python
from alphagenome_pt import deepmind_model

model = deepmind_model(load_state=True)
```

4.2.2. *Published Architecture with Custom Metadata*

The published architecture and its shared checkpoint parameters can be combined with custom metadata for different organisms and/or output tracks (useful for finetuning). Setting `organisms=False` and `heads=False` retains the initialization of the custom organism-

specific and output-track-specific parameters while loading the remaining compatible checkpoint state:

```
from alphagenome_pt import Metadata, deepmind_model

metadata = Metadata({
    "organisms": ["dragon"],
    "heads": {
        "rna_seq": {
            "num_tracks": [2],
            "means": [[2.1, 0.8]],
        },
    },
})

model = deepmind_model(
    metadata=metadata,
    load_state=True,
    organisms=False,
    heads=False,
)
```

Prefix loading and explicit mappings of parameters are also supported.

4.2.3. *Custom Architecture*

The metadata and architecture can be customized (useful for training from scratch):

```
from alphagenome_pt import AlphaGenome, AlphaGenomeConfig, Metadata

metadata = Metadata({
    "organisms": ["dragon"],
    "heads": {
        "rna_seq": {
            "num_tracks": [2],
            "means": [[2.1, 0.8]],
        },
    },
})

model = AlphaGenome(
    AlphaGenomeConfig(
        max_seq_len=8_192,
        num_channels=96,
        metadata=metadata,
    )
)
```

When training from scratch, our design philosophy is to let users control the most important model parameters while still defaulting toward the original AlphaGenome architecture. For example, users can directly set parameters such as `num_channels` and `max_seq_len`, which control the model width and input sequence length. Other hyperparameters, including `channel_increment`, `qk_head_dim`, `v_head_dim`, `pair_channels`, `num_splice_sites`, and `splice_site_channels`, can also be specified manually. However, when these values are not provided, `alphagenome-pt` infers defaults that keep their proportions consistent with the published model. This allows users to scale the model up or down via `num_channels` and `max_seq_len` without needing to manually adjust every dependent hyperparameter.

```
from alphagenome_pt import AlphaGenomeConfig, deepmind_config

official_cfg = deepmind_config()
small_cfg = AlphaGenomeConfig(max_seq_len=8_192, num_channels=96)

# Same proportions
assert small_cfg.channel_increment / small_cfg.num_channels == official_cfg.channel_increment /
official_cfg.num_channels == 1 / 6
assert small_cfg.qk_head_dim / small_cfg.num_channels == official_cfg.qk_head_dim /
official_cfg.num_channels == 1 / 6
assert small_cfg.v_head_dim / small_cfg.num_channels == official_cfg.v_head_dim /
official_cfg.num_channels == 1 / 4
assert small_cfg.pair_channels / small_cfg.num_channels == official_cfg.pair_channels /
official_cfg.num_channels == 1 / 6
assert small_cfg.num_splice_sites / small_cfg.max_seq_len == official_cfg.num_splice_sites /
official_cfg.max_seq_len == 1 / 2048
assert small_cfg.splice_site_channels / small_cfg.num_channels == official_cfg.splice_site_channels /
official_cfg.num_channels == 1
```

4.2.4. *Metadata*

Metadata specifies the organism and output heads for the model architecture. Real workflows will need to calculate the true means of nonzero target values, but dummy values are used here for simplicity.[3]

[3] Masked language modeling and sequence-to-function heads are not technically mutually exclusive. However, our example presents them as separate options because combining them is uncommon (masked language modeling corrupts the input sequence that is also used for functional prediction).

```python
from alphagenome_pt import AlphaGenome, AlphaGenomeConfig, Metadata, synthetic_batch

# Dummy Example: replace with actual nonzero means
def make_means(num_tracks):
    max_t = max(num_tracks)
    return [[1.0] * max_t for _ in num_tracks]

# Option 1: masked language modeling
mlm_metadata = Metadata({
    "organisms": ["human", "mouse"],
    "heads": {
        "masked_language_modeling": {},
    },
})

# Option 2: sequence-to-function prediction
functional_metadata = Metadata({
    "organisms": ["human", "mouse"],
    "heads": {
        # Gene expression / transcriptional activity
        "rna_seq": {
            "num_tracks": [10, 8],
            "means": make_means([10, 8]),
        },
        "cage": {
            "num_tracks": [6, 4],
            "means": make_means([6, 4]),
        },
        "procap": {
            "num_tracks": [3, 2],
            "means": make_means([3, 2]),
        },

        # Chromatin accessibility / regulatory binding
        "atac": {
            "num_tracks": [5, 4],
            "means": make_means([5, 4]),
        },
        "dnase": {
            "num_tracks": [5, 5],
            "means": make_means([5, 5]),
        },
        "chip_tf": {
            "num_tracks": [7, 6],
            "means": make_means([7, 6]),
        },
        "chip_histone": {
            "num_tracks": [4, 3],
            "means": make_means([4, 3]),
        },

        # Splicing
        "splice_sites_classification": {
            "num_tracks": [5, 5],
        },
        "splice_sites_usage": {
            "num_tracks": [4, 3],
        },
        "splice_sites_junction": {
            "num_tissues": [4, 3],
        },

        # Contact maps
        "contact_maps": {
            "num_tracks": [2, 2],
        },

    },
})

# Build a small model and a synthetic batch containing
# targets for every head present in the metadata
S = 2048
meta = functional_metadata  # or mlm_metadata
```

```
model_cfg = AlphaGenomeConfig(
    max_seq_len=S,
    num_channels=96,
    metadata=meta,
)
model = AlphaGenome(model_cfg)
data = synthetic_batch(
    meta,
    batch_size=4,
    seq_len=S,
    num_splice_sites=model_cfg.num_splice_sites,
)

# Calculate and backpropagate on the loss
model.train()
output = model(
    data,
    mode="loss",
    return_predictions=True,
    return_embeddings=True,
)
output.total.backward()

print(f"total loss: {output.total.item():.4f}")
```

4.3. Training

Load the published checkpoint and calculate the built-in losses when providing a `DataBatch` with targets:

```
import torch
from alphagenome_pt import deepmind_model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = deepmind_model(load_state=True, device=device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)
model.train()

for batch in train_loader:
    batch.to(device)
    optimizer.zero_grad(set_to_none=True)

    output = model(batch, mode="loss")
    output.total.backward()
    optimizer.step()
```

Alternatively, predictions and embeddings remain differentiable during direct model calls and can be used to define a custom objective:

```
model.train()

for batch in train_loader:
    batch.to(device)
    optimizer.zero_grad(set_to_none=True)

    predictions, embeddings = model(batch, return_embeddings=True)
    loss = custom_loss(predictions, embeddings, batch)
    loss.backward()
    optimizer.step()
```

4.4. Checkpoint Loading

`deepmind_model()` can load the all-folds model or folds 0–3 from the converted checkpoints hosted on Hugging Face:

```python
from alphagenome_pt import deepmind_model

model = deepmind_model(
    load_state=True,
    fold="all_folds",
    device="cuda",
    map_location="cpu",
)
```

Argument	Default	Behavior
`load_state`	`False`	Whether to load converted checkpoint parameters
`fold`	`"all_folds"`	Selects `"all_folds"` or `"fold_0"` through `"fold_3"`
`local_dir`	`None`	Optional local directory for cached or downloaded metadata and checkpoint files
`repo_id`	`"RylieWeaver/alphagenome-pytorch"`	Hugging Face repository containing the checkpoint artifacts
`repo_dir`	`None`	Repository subdirectory (resolves to `v{package_version}` when omitted)
`map_location`	`"cpu"`	Device used to deserialize the checkpoint
`device`	`"cpu"`	Final device receiving the model

5. Validation

5.1. Checkpoint Numerical Equivalence

We evaluated full-model numerical equivalence between `alphagenome-pt` and the official JAX implementation using the published `all_folds` checkpoint and its converted PyTorch counterpart. Using deterministic 131,072-bp sequences from human and mouse chromosome 1, we compared the resulting embeddings, predictions, and losses under the DeepMind mixed-precision, FP32, and FP64 policies.

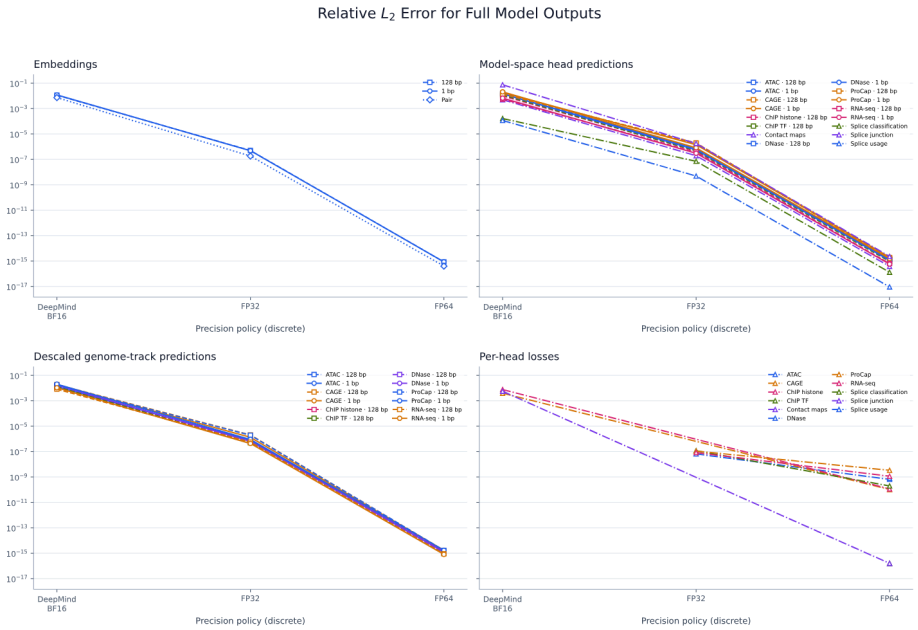


Figure 3. *Relative L_2 differences between the PyTorch and JAX full-model outputs under the DeepMind mixed-precision, FP32, and FP64 policies. Exact-zero differences are omitted from the plot.*

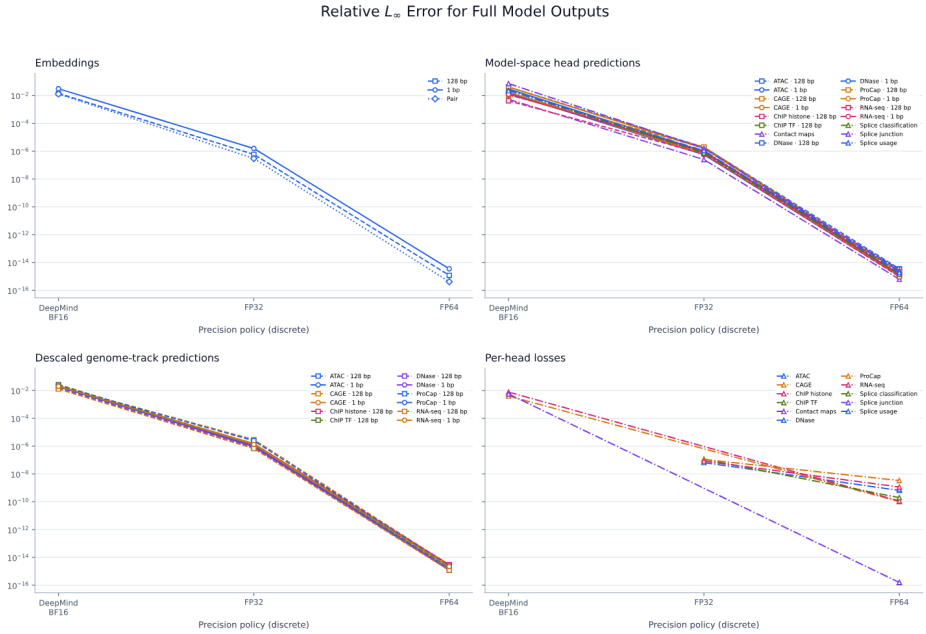


Figure 4. *Relative L_∞ differences between the PyTorch and JAX full-model outputs under the DeepMind mixed-precision, FP32, and FP64 policies. Exact-zero differences are omitted from the plot.*

All tested embeddings, model-space predictions, descaled predictions, and per-head losses satisfy the relative L_2 and L_∞ tolerances defined for their respective precision policies. The differences decrease from the DeepMind mixed-precision policy to FP32 to FP64, indicating that the remaining discrepancies are primarily due to floating-point precision and accumulated rounding error.

5.2. Model Size

The published AlphaGenome configuration contains 450.5 million trainable parameters, including its 11 output heads and organism-specific parameters. `alphagenome-pt` exposes hyperparameters for adjusting model size, with `num_channels` serving as the primary control for other architecture dimensions when they are not specified explicitly.

Configuration	`num_channels`	Output heads	Trainable parameters
Published AlphaGenome	768	11	450.5 M
Custom, headless	64	0	2.8 M
Custom, headless	128	0	11.3 M
Custom, headless	256	0	44.6 M
Custom, headless	512	0	179.2 M
Custom, headless	768	0	403.9 M

All configurations in this table use a maximum sequence length of 1,048,576 bp. The custom configurations use one organism and no output heads because these components depend on the intended training application. Excluding them isolates how the shared architecture —and therefore its parameter count—scales with `num_channels`.

5.3. Inference and Training Throughput

We measured latency and peak allocated GPU memory for the published architecture during inference and training. Training measurements used power-of-two sequence lengths from 2^{11} (2,048) to 2^{17} (131,072) bp, while inference measurements also included 2^{18} (262,144) bp. Embedding runs excluded output heads, whereas prediction runs included all 11 published heads. We used `num_splice_sites=512` for every sequence length, in line with the published model configuration.

Throughput benchmark configuration

Benchmarks used one NVIDIA A100-SXM4-80GB GPU, batch size 1, the DeepMind precision policy, PyTorch 2.6.0 with CUDA 12.4, and `alphagenome-pt` 0.4.0. Each measurement reports the mean of 100 timed iterations following 20 warm-up iterations. Training measurements used AdamW with a learning rate of 3×10^{-5} and included the forward pass, loss calculation, backward pass, and optimizer step. The model parameters were randomly initialized.

Training used synthetic batches. Embedding training minimized mean squared error against synthetic embedding targets, while all-head prediction training used the package's built-in losses with synthetic targets.

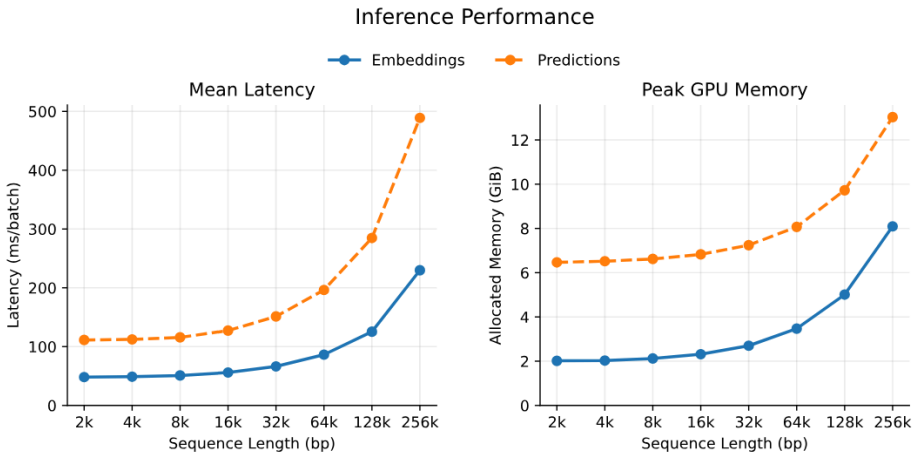


Figure 5. *Mean inference latency and peak allocated GPU memory as functions of sequence length for embedding-only and all-head prediction inference.*

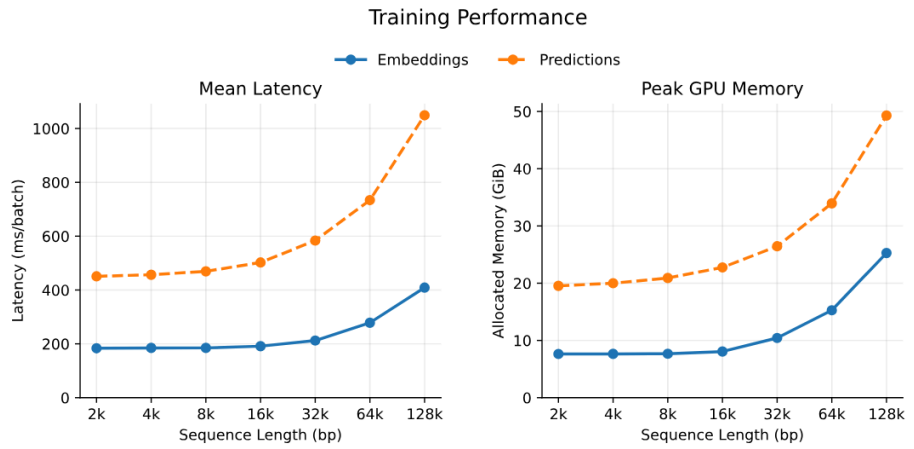


Figure 6. *Mean training-step latency and peak allocated GPU memory as functions of sequence length for embedding-only and all-head prediction training.*

5.4. Training Curves

As training examples, we optimized the published architecture from both random and released parameters on masked language modeling and RNA-seq prediction using human chromosome 1.

Training configuration

Each run used 16,384-bp sequences, a batch size of 8, a learning rate of 3×10^{-5}, the DeepMind precision policy, seed 42, and 1,000 training steps. Validation metrics were evaluated every 10 steps over 10 batches.

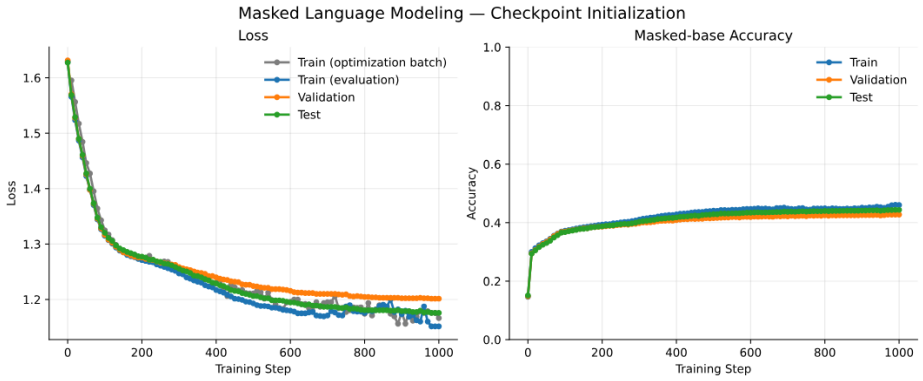


Figure 7. *Masked language modeling training, validation, and test metrics when initialized from the converted checkpoint.*

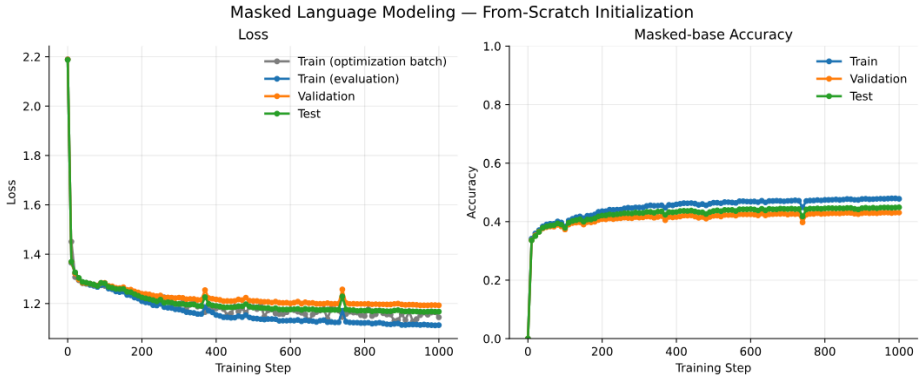


Figure 8. *Masked language modeling training, validation, and test metrics when initialized from random parameters.*

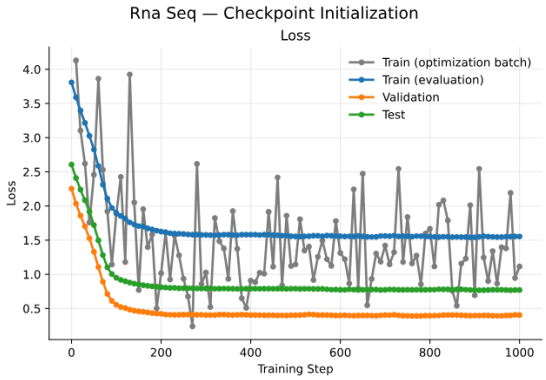


Figure 9. *RNA-seq training, validation, and test loss when initialized from the converted checkpoint.*

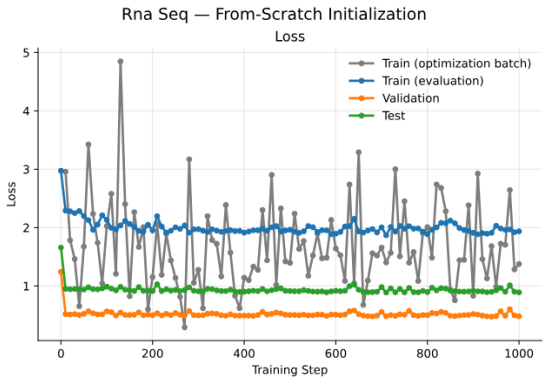


Figure 10. *RNA-seq training, validation, and test loss when initialized from random parameters.*

All four experiments completed successfully and showed an initial decrease in loss, illustrating the software's ability to train from both checkpoint and random initialization for MLM and RNA-seq prediction tasks.

6. Limitations and Related Work

The first main area of limitation in `alphagenome-pt` is the absence of sequence-parallelism execution, which Google DeepMind used to train AlphaGenome on very long sequences. A second limitation is that `alphagenome-pt` does not provide a preprocessing pipeline for functional genomics data. However, this is an intentional scope choice because the proper storage formats can differ between large-scale HPC training and smaller experiments. Rather than assuming a particular file format or data-loading strategy, `alphagenome-pt` assumes that users yield tensorized or sequence data to the model, however that is done. The package focuses on the model rather than prescribing a universal data-processing workflow.

The most closely related software is the official AlphaGenome release from Google DeepMind (Link1, Link2). That implementation remains the reference implementation for the model and released checkpoints, but it is written in JAX, whereas `alphagenome-pt` is motivated by the complementary goal of making the model trainable in PyTorch. Several unofficial PyTorch implementations of AlphaGenome also exist. The `genomicsxai/alphagenome-pytorch` package is a strong and polished PyTorch port, with PyTorch-converted checkpoint loading, named outputs, example notebooks, fine-tuning utilities, and reported numerical parity with the original JAX model. `alphagenome-pt` shares many capabilities with `genomicsxai/alphagenome-pytorch` but places more emphasis on flexible model training, as shown by its flexible checkpoint loading and exposed architecture hyperparameters.

A previous PyTorch implementation by Phil Wang (also known as lucidrains on GitHub), Miquel Anglada-Girotto, and Xinming Tu is also available at lucidrains/alphagenome.

Relative to these related projects, the goal of `alphagenome-pt` is not to replace the official implementation or provide the most complete inference interface. Instead, the goal is to provide a clear and maximally flexible PyTorch training implementation for users who want to adapt AlphaGenome to new datasets, species, and biological contexts while still remaining faithful to the original architecture.

7. Conclusions and Future Work

We presented `alphagenome-pt`, an open-source PyTorch implementation of AlphaGenome for training and fine-tuning. The goal of the package is to make it easier for researchers to train AlphaGenome models on their own datasets, species, and biological contexts without reimplementing the model themselves.

By lowering the software barrier to AlphaGenome training, `alphagenome-pt` aims to make sequence-to-function modeling more accessible beyond the released human and mouse models. More broadly, we hope this helps researchers use computational models to generate and test biological hypotheses faster.

Future work will focus on implementing distributed sequence-parallel training to support longer sequences and larger-scale HPC training.

8. Generative AI Disclosure

Portions of this work were assisted using ChatGPT and Codex. The tools were used for drafting and revising code in the package and text for this proceedings submission. All outputs were reviewed, verified, and revised by the author(s), who take full responsibility for the accuracy and integrity of the final content.

References

[1] G. Brixi *et al.*, "Genome modelling and design across all domains of life with Evo 2," *Nature*, vol. 652, no. 8112, pp. 1349–1361, 2026, doi: 10.1038/s41586-026-10176-5.

[2] E. Nguyen *et al.*, "Sequence modeling and design from molecular to genome scale with Evo." [Online]. Available: https://www.biorxiv.org/content/10.1101/2024.02.27.582234v1

[3] E. Nguyen *et al.*, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023, pp. 43177–43201. doi: 10.52202/075280-1872.

[4] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. Davuluri, and H. Liu, "DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome." [Online]. Available: http://arxiv.org/abs/2306.15006

[5] Y. Ji, Z. Zhou, H. Liu, and R. V. Davuluri, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome," *Bioinformatics*, vol. 37, no. 15, pp. 2112–2120, 2021, doi: 10.1093/bioinformatics/btab083.

[6] H. Dalla-Torre *et al.*, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics," *Nature Methods*, vol. 22, no. 2, pp. 287–297, 2025, doi: 10.1038/s41592-024-02523-z.

[7] H. Dalla-Torre *et al.*, "The Nucleotide Transformer: Building and Evaluating Robust Foundation Models for Human Genomics." [Online]. Available: https://www.biorxiv.org/content/10.1101/2023.01.11.523679v1

[8] J. Zhai *et al.*, "PlantCAD2: A Long-Context DNA Language Model for Cross-Species Functional Annotation in Angiosperms." [Online]. Available: https://www.biorxiv.org/content/10.1101/2025.08.27.672609v1

[9] Ž. Avsec *et al.*, "Advancing regulatory variant effect prediction with AlphaGenome," *Nature*, vol. 649, no. 8099, pp. 1206–1218, 2026, doi: 10.1038/s41586-025-10014-0.

[10] Ž. Avsec *et al.*, "Effective gene expression prediction from sequence by integrating long-range interactions," *Nature Methods*, vol. 18, no. 10, pp. 1196–1203, 2021, doi: 10.1038/s41592-021-01252-x.

[11] J. Zhou and O. G. Troyanskaya, "Predicting effects of noncoding variants with deep learning-based sequence model," *Nature Methods*, vol. 12, no. 10, pp. 931–934, 2015, doi: 10.1038/nmeth.3547.

[12] D. R. Kelley, Y. A. Reshef, M. Bileschi, D. Belanger, C. Y. McLean, and J. Snoek, "Sequential regulatory activity prediction across chromosomes with convolutional neural networks," *Genome Research*, vol. 28, no. 5, pp. 739–750, 2018, doi: 10.1101/gr.227819.117.

[13] J. Linder, D. Srivastava, H. Yuan, V. Agarwal, and D. R. Kelley, "Predicting RNA-seq coverage from DNA sequence as a unifying model of gene regulation," *Nature Genetics*, vol. 57, no. 4, pp. 949–961, 2025, doi: 10.1038/s41588-024-02053-6.

[14] K. Jaganathan *et al.*, "Predicting Splicing from Primary Sequence with Deep Learning," *Cell*, vol. 176, no. 3, pp. 535–548.e24, 2019, doi: 10.1016/j.cell.2018.12.015.

[15] T. Zeng and Y. I. Li, "Predicting RNA splicing from DNA sequence using Pangolin," *Genome Biology*, vol. 23, no. 1, p. 103, 2022, doi: 10.1186/s13059-022-02664-4.

[16] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs." [Online]. Available: http://github.com/jax-ml/jax

[17] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[18] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.

[19] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?." [Online]. Available: http://arxiv.org/abs/1411.1792

[20] J. Howard and S. Ruder, "Universal Language Model Fine-tuning for Text Classification." [Online]. Available: http://arxiv.org/abs/1801.06146

[21] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." [Online]. Available: http://arxiv.org/abs/1810.04805

[22] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., Cham, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.

[23] A. Vaswani *et al.*, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf